Laporan Tugas Besar 1

IF2211 Strategi Algoritma

Pemanfaatan Algoritma Greedy dalam Pembuatan Bot Permainan Battlecode

2025

Kelompok 40 “beeOL”

Anggota:

Reinhard Alfonzo Hutabarat	(13524056)
Arghawisesa Dwinanda Arham	(13524100)
Amanda Aurellia Salsabilla	(13524131)

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

BANDUNG

2025

PERNYATAAN

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Reinhard Alfonzo Hutabarat

Arghawisesa Dwinanda Arham

Amanda Aurellia Salsabilla

DAFTAR ISI

PERNYATAAN **2**
DAFTAR ISI **3**
BAB 1 ..... **5**
BAB 2 ..... **6**
2.1 Dasar Teori 6
2.2 Cara Kerja Program 6
BAB 3 ..... **8**
3.1. Mapping Persoalan 8
3.2. Eksplorasi Algoritma Greedy
Eksplorasi 3 alternatif solusi greedy yang mungkin dipilih dalam persoalan battlecode. (setiap alternatif harus memiliki heuristic yang berbeda) 8
3.2.1. Algoritma Main Bot
Pergerakan bot dalam permainan ini dikendalikan oleh algoritma Greedy yang mengevaluasi delapan arah mata angin di sekitar robot pada setiap gilirannya. Keputusan pergerakan diambil berdasarkan kalkulasi skor heuristik sesaat tanpa melakukan simulasi langkah ke depan. Berikut adalah rincian strategi untuk setiap elemen, ... 8
3.2.2. Algoritma Bot Alternatif 1 10
3.2.3. Algoritma Bot Alternatif 2 11
3.3. Analisis Efisiensi dan Efektivitas 13
3.4. Strategi Greedy Pilihan 13
BAB 4 ..... **14**
4.1 Implementasi 14
4.1.1 Main Bot: Alfonzo 14
4.1.1.1 Pseudocode 14
4.1.1.2 Struktur Data 28
4.1.2 Bot Alternatif 1: Argha 32
4.1.2.1 Pseudocode 32
4.1.2.2 Struktur Data 44
4.1.3 Bot Alternatif 2: Aurel 44
4.1.3.1 Pseudocode 44
4.1.3.2 Struktur Data 55
4.2 Pengujian 59
4.2.1 Bot Utama vs Bot Alternatif 1 59
4.2.1.1 Kasus 1: [Nama Map] 59
4.2.1.2 Kasus 2: [Nama Map] 59
4.2.1.3 Kasus 3: [Nama Map] 59
4.2.1.4 Kasus 4: [Nama Map] 59

4.2.1.5 Kasus 5: [Nama Map]..... 59
4.2.2 Bot Utama vs Bot Alternatif 2..... 59
4.2.2.1 Kasus 1: [Nama Map]..... 59
4.2.2.2 Kasus 2: [Nama Map]..... 59
4.2.2.3 Kasus 3: [Nama Map]..... 59
4.2.2.4 Kasus 4: [Nama Map]..... 59
4.2.2.5 Kasus 5: [Nama Map]..... 59
4.3 Analisis Hasil Pengujian..... 59
BAB 5..... 60
5.1 Kesimpulan..... 60
5.2 Saran..... 60
LAMPIRAN.....62
DAFTAR PUSTAKA..... 63

BAB 1
DESKRIPSI TUGAS

Tugas Besar 1 IF2211 Strategi Algoritma berfokus pada pemanfaatan algoritma *Greedy* dalam pembuatan bot untuk permainan Battlecode 2025. Battlecode adalah permainan strategi waktu nyata (*real-time strategy*) di mana pemain harus menulis kode untuk entitas otonom. Dalam tugas ini, pemain dituntut untuk mengelola pasukan robot secara strategis agar dapat bekerja sama mengalahkan tim lawan.

Melalui tugas ini, mahasiswa ditugaskan untuk mengimplementasikan dan mengasah kemampuan terkait kecerdasan buatan, penelusuran jalur (*pathfinding*), algoritma terdistribusi, dan komunikasi antar-unit. Namun, terdapat batasan sumber daya komputasi (*bytecode limit*) pada setiap gilirannya, sehingga mahasiswa dituntut untuk merencanakan strategi seefisien mungkin, melakukan optimasi program, serta mengambil kompromi keputusan sesuai batasan yang ada. Tujuan akhirnya adalah merancang sebuah bot dengan pendekatan algoritma *Greedy* yang nantinya akan dipertandingkan satu sama lain dalam lingkungan kompetitif.

BAB 2
LANDASAN TEORI

2.1 Dasar Teori

Algoritma *Greedy* merupakan salah satu pendekatan penyelesaian masalah optimasi yang beroperasi secara langkah demi langkah (*step-by-step*). Prinsip fundamental dari algoritma ini adalah mengambil pilihan terbaik yang tersedia pada saat itu (*local optimum*) dengan harapan bahwa akumulasi dari keputusan-keputusan lokal tersebut akan mengarah pada solusi yang optimal secara keseluruhan (*global optimum*). Algoritma ini bersifat miopik, artinya ia tidak mempertimbangkan konsekuensi jangka panjang dari pilihan yang telah diambil, melainkan hanya berfokus pada keuntungan maksimal atau kerugian minimal yang ada di depan mata pada setiap langkahnya.

Dalam penyusunan strukturnya, algoritma *Greedy* dibangun oleh enam elemen utama:

- Himpunan Kandidat: Kumpulan data atau elemen persoalan yang bisa dipilih pada setiap langkah untuk membentuk sebuah solusi.
- Himpunan Solusi: Kumpulan kandidat terpilih yang secara bertahap membentuk solusi akhir.
- Fungsi Solusi: Menentukan apakah solusi sudah lengkap.
- Fungsi Seleksi: Aturan heuristik yang digunakan untuk mengevaluasi dan memilih kandidat terbaik dari himpunan kandidat pada saat itu.
- Fungsi Kelayakan: Pemeriksaan kondisional untuk memastikan apakah kandidat yang dipilih oleh fungsi seleksi tidak melanggar batasan-batasan dari masalah.
- Fungsi Objektif: Nilai atau target akhir yang ingin dimaksimalkan atau diminimalkan dari penyelesaian persoalan tersebut.

2.2 Cara Kerja Program

Dalam implementasinya pada sebuah program, algoritma *Greedy* bekerja melalui sebuah siklus perulangan sederhana. Pada fase awal, himpunan solusi berada dalam keadaan kosong. Di setiap iterasinya, program akan mengevaluasi seluruh kandidat yang tersedia menggunakan Fungsi Seleksi untuk menentukan kandidat dengan bobot atau skor paling menguntungkan.

Setelah kandidat terbaik dipilih, program akan memvalidasinya menggunakan Fungsi Kelayakan. Jika kandidat tersebut valid dan tidak menyalahi aturan batasan masalah, maka ia akan dikeluarkan dari himpunan kandidat dan dimasukkan ke dalam himpunan solusi. Sebaliknya, jika tidak layak, kandidat tersebut dibuang secara permanen.

Selanjutnya, program akan menggunakan Fungsi Solusi untuk mengevaluasi himpunan solusi yang baru saja diperbarui. Fungsi ini bertugas secara spesifik untuk menentukan apakah himpunan solusi saat ini sudah lengkap dan berhasil memenuhi kriteria penyelesaian akhir dari permasalahan. Proses perulangan ini akan terus berjalan hingga Fungsi Solusi menyatakan bahwa solusi sudah berstatus lengkap, atau berhenti ketika himpunan kandidat telah habis dievaluasi seluruhnya.

BAB 3
APLIKASI STRATEGI GREEDY

3.1. Mapping Persoalan

Berikut adalah pemetaan elemen-elemen algoritma *Greedy* terhadap mekanik persoalan Battlecode 2025:

3.1.1. Himpunan Kandidat

Himpunan kandidat merepresentasikan seluruh kemungkinan aksi yang dapat dilakukan oleh sebuah robot pada satu giliran tertentu. Tergantung pada jenis unitnya (Tower, Soldier, Splasher, atau Mopper), himpunan kandidat ini mencakup:

- Kandidat pergerakan: Delapan arah mata angin di sekitar robot.
- Kandidat serangan/pengecatan: Seluruh koordinat petak yang berada di dalam radius aksi robot.
- Kandidat pembangunan/produksi: Titik-titik reruntuhan untuk mendirikan menara, atau arah *spawn* untuk memproduksi robot baru.
- Kandidat transfer logistik: Unit-unit sekutu di sekitar yang dapat menerima distribusi cat atau *chips*.

3.1.2. Fungsi Kelayakan

Fungsi ini bertugas memvalidasi apakah sebuah kandidat aksi dari himpunan kandidat sah untuk dieksekusi berdasarkan aturan permainan. Fungsi kelayakan akan mengevaluasi parameter status robot dan lingkungannya, seperti:

- Apakah status jeda (*action/movement cooldown*) robot berada di bawah angka 10?
- Apakah kuantitas cat atau *chips* yang dimiliki mencukupi untuk melakukan aksi tersebut?
- Apakah petak tujuan pergerakan tidak terhalang oleh tembok (*wall*), reruntuhan, atau robot lain?
- Apakah target koordinat serangan atau pembangunan berada di dalam batas peta permainan yang sah?

Kandidat aksi yang melanggar batasan-batasan ini dinyatakan tidak layak dan akan dikeluarkan dari pertimbangan.

3.1.3. Fungsi Seleksi

Fungsi seleksi adalah inti dari heuristik *Greedy* yang memberikan penilaian (pembobotan skor) terhadap setiap kandidat aksi yang telah dinyatakan layak. Evaluasi ini didasarkan pada perhitungan keuntungan instan terbesar yang bisa didapat pada giliran tersebut. Contoh penerapannya meliputi:

- Memberikan skor tinggi pada kandidat arah pergerakan yang membawa robot mendekati area yang belum terpetakan atau area strategis musuh.
- Memberikan skor tinggi pada target tembakan cat (*splash/attack*) yang mengenai kepadatan tanah kosong atau cat musuh paling banyak dalam satu kali eksekusi.
- Memberikan skor tinggi pada target musuh yang memiliki *Hitpoints* (HP) paling rendah agar dapat segera dimusnahkan.

Kandidat dengan skor evaluasi tertinggi akan dipilih sebagai aksi terbaik saat itu.

3.1.4. Himpunan Solusi

Himpunan solusi merupakan kumpulan aksi terpilih yang secara nyata dieksekusi oleh robot ke dalam *game engine*. Dalam skala mikro (satu giliran), himpunan solusi adalah aksi tunggal (misalnya: bergerak ke Utara, lalu menembak koordinat X,Y) yang berhasil dijalankan oleh robot tersebut. Dalam skala makro, himpunan solusi adalah akumulasi dari seluruh keputusan eksekusi dari seluruh robot sekutu dari ronde pertama hingga permainan berakhir.

3.1.5. Fungsi Objektif

Fungsi objektif adalah tujuan matematis yang ingin dicapai dari pengambilan keputusan *Greedy* tersebut. Pada Battlecode 2025, fungsi objektif utamanya adalah memaksimalkan persentase jumlah petak di peta yang memiliki warna cat sekutu (*paint coverage*). Selain itu, terdapat fungsi objektif sekunder berupa minimasi, yaitu meminimalkan total HP dan dominasi cat dari pasukan musuh.

3.1.6. Fungsi Solusi

Fungsi solusi bertindak sebagai kondisi pemberhentian yang mengevaluasi apakah himpunan solusi yang telah dikumpulkan sudah mencapai target akhir permasalahan. Pada tingkat permainan (makro), fungsi solusi dinilai lengkap apabila kondisi kemenangan telah terpenuhi, yaitu ketika total area cat sekutu telah mencapai atau melebihi 70% dari luas peta yang dapat dilewati, atau ketika seluruh menara dan unit musuh telah hancur. Pada tingkat giliran (mikro), fungsi solusi terpenuhi ketika robot telah kehabisan batas *bytecode* atau batas aksinya pada ronde tersebut, sehingga program mengembalikan kendali kepada sistem.

3.2. Eksplorasi Algoritma Greedy

Eksplorasi 3 alternatif solusi greedy yang mungkin dipilih dalam persoalan battlecode. (setiap alternatif harus memiliki heuristic yang berbeda)

3.2.1. Algoritma Main Bot

Pergerakan bot dalam permainan ini dikendalikan oleh algoritma *Greedy* yang mengevaluasi delapan arah mata angin di sekitar robot pada setiap gilirannya. Keputusan pergerakan diambil berdasarkan kalkulasi skor heuristik sesaat tanpa melakukan simulasi langkah ke depan. Berikut adalah rincian strategi untuk setiap elemen,

- Tower

  Tower bertindak sebagai titik tumpu pertahanan dan produksi tim. Ia menyerang robot musuh di dalam jangkauan yang memiliki HP paling rendah agar musuh lebih cepat mati. Tower secara proaktif mentransfer cat ke unit sekutu di sekitarnya yang kapasistas catnya berada di bawah 33%, bertindak sebagai stasiun pengisian ulang. Selain itu, ia juga memproduksi unit dengan komposisi yang bergantung pada fase ronde dan luas peta. Pada peta kecil, produksi difokuskan pada Splasher sejak awal untuk *instant war*. Pada peta besar, fase awal didominasi oleh Soldier untuk eksplorasi, lalu perlahan beralih ke Splasher di pertengahan permainan. Terakhir, tower ditugaskan untuk mengevaluasi petak sekitar untuk menempatkan robot baru di arah yang minim kerumunan sekutu tetapi dekat dengan jejak cat musuh.

- Soldier

  Soldier bertugas sebagai sang eksplorator terpintar dan koordinator lapangan. Ia secara otomatis menentukan sektor tujuannya berdasarkan ID robot untuk memastikan eksplorasi wilayah yang merata di awal permainan. Lalu, ia melakukan pembangunan menara dengan mengevaluasi reruntuhan terdekat dengan prioritas pada reruntuhan yang letaknya mendekati titik tengah peta dan basis musuh. Jika soldier melihat musuh sedang membangun pola menara di kejauhan, ia akan menembakkan cat ke area tersebut untuk merusak pola dan membuang waktu musuh. Selain itu, ia mengevaluasi petak di area serangannya. Menara musuh memiliki prioritas mutlak (skor 3000/jarak), disusul cat musuh yang menempel pada batas cat sekutu (skor 180), cat musuh biasa (skor 80), dan terakhir tanah kosong (skor 15),
- Splasher

  Sang cendekiawan yang katanya terkuat dari yang terkuat demi mencapai objektif penguasaan 70% wilayah. Splasher menghitung jumlah cat musuh di sekitarnya. Jika ia melihat terlalu banyak cat musuh, ia masuk ke *Reclaim Mode*, di mana prioritas utamanya adalah menimpa cat lawan daripada mengecat tanah kosong. Ia menilai kualitas tembakan pada area 3x3. Cat musuh memberikan (+10) poin dan tanah kosong memberikan (+5) poin. Tembakan Splash hanya akan dieksekusi jika total poin di area target menembus ambang batas efisiensi tertentu agar cat yang mahal tidak terbuang sia-sia.
- Mopper

  Mopper bertindak sebagai *support* garis terdepan dengan mobilitas yang tinggi. Ia mengevaluasi arah ayunan pel dan akan mengeksekusi ayunan pada arah yang mengenai jumlah robot musuh terbanyak. Mopper juga akan mencari sekutu di sekitarnya yang kapasitas catnya kritis (<35%) dan menyumbangkan cat dari badannya sendiri. Terakhir, ia menghapus cat musuh satu per satu di area jangkauannya, dengan bonus skor tinggi (+25) jika ada robot musuh yang berdiri tepat di atas cat tersebut.

- Fungsi Helper

  Navigasi robot dieksekusi melalui dua fungsi utama yang mengevaluasi delapan arah mata angin menggunakan pembobotan skor murni.

  - smartMoveTo: Fungsi navigasi untuk bergerak menuju target yang sudah pasti, misal mendekati runtuhan atau pulang ke tower. Fungsi ini akan mencari arah dengan skor minimum dan mempertahankan momentum langkah dengan pengurangan skor (-25) agar robot dapat meluncur mulus saat membentur tembok. Fungsi ini juga menghindari petak musuh dengan penalti besar (+320) jika sisa cat robot di bawah 50% dan mencegah perulangan dengan memberi penalti raksasa (+4000) pada petak yang ada di riwayat visitedTiles.

  - fluidExploreMove: Fungsi navigasi untuk robot yang sedang berekspansi bebas tanpa target titik absolut. Berbeda dengan smartMove, fungsi ini memilih arah dengan skor tertinggi dan bergerak dari atas cat sekutu menuju tanah kosong/musuh diberi bonus (+750) agar robot terdorong ke batas terluar wilayah. Jika sensor menangkap keberadaan musuh, robot akan mendekati sekutu (+200 per teman) untuk mengeroyok. Jika area aman, robot akan saling menjauh (-140 per teman) agar penyebaran cat lebih optimal. Terakhir, fungsi ini memberikan penalti jarak ke menara terdekat yang semakin membesar apabila persentase cat di badan robot semakin menipis, mencegah robot mengembara terlalu jauh dan mati sebelum sempat pulang.

3.2.2. Algoritma Bot Alternatif 1

Bot ini memaksimalkan efisiensi pada penggunaan cat yang ada, di mana bot akan memaksimalkan coverage per unit cat yang ada untuk dihabiskan. Prinsip utamanya adalah mengandalkan jalur ally teritory (berbiaya 0) dibandingkan neutral (berbiaya 1) ataupun enemy (berbiaya 2). Berikut merupakan penjelaskan konsep greedy pada tiap unit.

- Tower

  Tower akan melakukan seleksi dalam melakukan spawn dengan perhitungan cat musuh di sekitar -robot team di sekitar . Terdapat 3 fase spawn di mana

pada ronde 0-100 adalah spawn soldier sepenuhnya, 100-300 spawn soldier 50%; splasher 40%; dan mopper 10%, dan ronde diatas 300 akan spawn splasher 60%; soldier 20%; dan mopper 20%. Kemudian jika cat musuh lebih dari 8 petak di sekitar maka akan dilakukan spawn mopper.

- Soldier

  Soldier akan bergerak dengan perhitungan skor. Skor dihitung dengan rumus (petak kosong * 12) - (biaya cat * 15) - (robot sekutu * 8) - penalty. Kemudian soldier hanya akan mengejar ruins jika distanceSquarednya kurang dari sama dengan 4 sehingga tidak meninggalkan perbatasan. Prioritas pewarnaan yang dilakukan adalah cat musuh dekat wilayah team (dengan skor 5), lalu cat musuh jauh (dengan skor 2), lalu petak kosong (dengan skor 1). Soldier akan melakukan refill catt jika catnya kurang dari 30% kapasitas yang ia punya.

- Mopper

  Mop swing hanya akan terjadi jika lebih dari sama dengan 2 musuh berjejer, hal ini dilakukan agar hemat cooldown 20 ronde. Kemudian mopper akan bergerak dengan skor, di mana ia akan memprioritaskan petak cat musuh (+20), bonus robot musuh di sekitar (+25), dan lookahead 1 petak (+10). Mopper juga akan melakukan transfer cat pada soldier jika soldier memiliki cat di bawah 50% dari kapasitasnya.

- Splasher

  Splasher akan bergerak menuju area dengan cat musuh terbanyak di sekitarnya. Splasher pun memiliki skor dengan rumus (4 * cat musuh) + (2 * petak kosong) - (2 * petak sekutu). Kemudian splasher juga akan melakukan pengisian ulang cat jika catnya kurang dari 25%.

- Fungsi Helper

  Kemudian terdapat beberapa fungsi pendukung yang saya akan buat juga seperti untuk menghitung biaya berdasarkan tipe cat, untuk navigasi cat yang hemat dengan meminimasi dsitance + biaya * 15 + visited penalty, anti loop dengan circular buffer pada 30 lokasi terakhir dan penalti 100 per

lokasi yang pernah dikunjungi, serta penyimpanan untuk mengingat lokasi tower yang berada di luar sensor range.

3.2.3. Algoritma Bot Alternatif 2

Bot ini memaksimalkan jumlah menara sekutu yang dibangun secepat mungkin di area yang belum tercover. Prinsip utamanya adalah semakin banyak menara yang tersebar di seluruh peta, semakin banyak spawn point baru yang muncul di area yang belum diwarnai, sehingga robot dapat mewarnai wilayah secara paralel dari berbagai titik sekaligus. Berikut penjelasan untuk setiap komponennya.

- Tower

  Tower melakukan spawn dengan probabilitas tetap sepanjang permainan: 40% soldier, 30% splasher, dan 30% mopper. Soldier diprioritaskan karena berperan sebagai pembangun menara utama. Terdapat mekanisme fallback ke soldier apabila chips tidak mencukupi untuk tipe yang dipilih (splasher membutuhkan 400 chips, mopper membutuhkan 300 chips). Spawn hanya dilakukan apabila cadangan cat $\geq$ 300 dan chips $\geq$ 500 sebagai buffer agar menara tidak kehabisan sumber daya secara total. Tower juga secara aktif menyerang musuh dalam jangkauan dan melakukan upgrade diri sendiri apabila kondisi memungkinkan.

- Soldier

  Soldier secara aktif mencari ruin yang paling jauh dari semua menara sekutu yang sudah ada menggunakan memori ruin (knownRuins). Heuristik greedy-nya adalah memilih ruin dengan nilai minDistanceToNearestTower terbesar, karena area tersebut merupakan wilayah yang paling membutuhkan spawn point baru. Soldier tidak hanya bereaksi terhadap ruin yang kebetulan terlihat, melainkan aktif menuju ruin yang tersimpan di memori meskipun berada di luar jangkauan pandang. Selama perjalanan menuju ruin, soldier mewarnai setiap petak yang diinjak untuk memaksimalkan coverage. Pengisian ulang cat dilakukan apabila cadangan turun di bawah 30% kapasitas.

- Mopper

Mopper berpatroli di perbatasan wilayah sekutu untuk menjaga area yang sudah diwarnai dari serangan musuh. Heuristik pergerakan mengutamakan petak musuh (+50) sebagai target utama pembersihan, diikuti petak sekutu (+30) sebagai zona patroli, dan petak kosong (+10) sebagai jalur alternatif. Prioritas aksi adalah mop swing untuk serangan area, diikuti transfer cat sebesar 20 unit ke robot sekutu yang cadangan catnya turun di bawah 20% kapasitas, kemudian penghapusan cat musuh dengan prioritas petak yang berbatasan langsung dengan wilayah sekutu.

- Splasher

  Splasher bertugas menebalkan wilayah di sekitar menara yang baru dibangun. Splasher menuju menara sekutu dengan jumlah petak belum diwarnai terbanyak di sekitarnya sebagai target pergerakan. Pemilihan titik pusat serangan AoE dilakukan secara greedy berdasarkan skor: +2 untuk setiap petak musuh dan +1 untuk setiap petak kosong dalam radius dampak serangan, sehingga splasher selalu menyerang area yang memberikan coverage terluas sekaligus merebut wilayah musuh. Pengisian ulang cat dilakukan apabila cadangan turun di bawah 30% kapasitas.

- Fungsi Helper

  Terdapat beberapa fungsi pendukung yang diimplementasikan. Fungsi moveToTarget digunakan untuk navigasi menuju tujuan spesifik dengan meminimasi jarak kuadrat ke target ditambah penalti kunjungan sebesar 1000 per petak yang baru diinjak. Fungsi exploreMap digunakan untuk eksplorasi tanpa tujuan spesifik dengan sistem skor berbasis kondisi petak: +100 untuk petak unknown, +50 untuk petak kosong, +25 untuk petak musuh, dan −75 untuk petak sekutu. Mekanisme anti-loop diimplementasikan menggunakan circular buffer pada 40 lokasi terakhir dengan penalti 1000 (navigasi) dan 150 (eksplorasi) per lokasi yang pernah dikunjungi. Selain itu, terdapat fungsi penyimpanan memori untuk mengingat lokasi menara sekutu dan ruin yang berada di luar jangkauan sensor robot.

3.3. Analisis Efisiensi dan Efektivitas

Analisis efisiensi dan efektivitas dari kumpulan alternatif solusi greedy yang dirumuskan

3.4. Strategi Greedy Pilihan

Strategi greedy yang dipilih dari 3 alternatif solusi greedy (yang akan diimplementasikan dalam program) beserta alasan dan pertimbangan pemilihan strategi tersebut.

BAB 4

IMPLEMENTASI DAN PENGUJIAN

4.1 Implementasi

4.1.1 Main Bot: Alfonzo

4.1.1.1 Pseudocode

Program Utama

```
PROGRAM RobotPlayer:

VARIABEL GLOBAL:
    turnCount       ← 0
    directions      ← [NORTH, NORTHEAST, EAST,
SOUTHEAST,
                        SOUTH, SOUTHWEST, WEST,
NORTHWEST]
    mapMemory       ← array[61][61]
    visitedTiles    ← array[60]
    visitedIndex    ← 0
    lastDir         ← null
    alliedTowers    ← array[30]
    towerCount      ← 0
    spawnCounter    ← 0
    mapWidth, mapHeight, mapArea ← -1
    isSmallMap      ← false
    isMediumMap     ← false
    mapCenter       ← null
    symmetryType    ← 0
    predictedEnemyBase ← null
    sectorTarget    ← null
    idleTimer       ← 0
    IDLE_THRESHOLD  ← 30
    consecutiveEnemyPaintSeen ← 0
    knownRuins      ← array[50]
    ruinCount       ← 0

FUNGSI run(rc):
    JIKA mapArea = -1:
```

```
        mapWidth  ← lebarPeta
        mapHeight ← tinggiPeta
        mapArea   ← mapWidth × mapHeight
        isSmallMap  ← (mapArea < 900)
        isMediumMap ← (mapArea >= 900 DAN mapArea <=
2500)
        mapCenter   ← (mapWidth / 2, mapHeight / 2)

    ULANGI SELAMANYA:
        turnCount ← turnCount + 1
        updateMapMemory(rc)
        updateTowerMemory(rc)
        updateKnownRuins(rc)
        detectSymmetry(rc)

       JIKA tipe robot = SOLDIER  → jalankan
runSoldier(rc)
       JIKA tipe robot = MOPPER   → jalankan
runMopper(rc)
JIKA tipe robot = SPLASHER → jalankan
runSplasher(rc)
       SELAIN ITU (tower)         → jalankan
runTower(rc)
```

Tower

```
FUNGSI runTower(rc):
    JIKA bisa upgrade tower saat ini:
        upgrade tower

    enemies ← semua robot musuh dalam jangkauan
sensor
    JIKA aksi siap DAN ada musuh:
        target ← musuh dengan HP terendah yang bisa
diserang
        JIKA target ada: serang target

    JIKA cat tower > 350:
        UNTUK setiap ally bukan tower dalam
```

```
jangkauan sensor:
            JIKA cat ally < 1/3 kapasitasnya:
                jumlah_beri ← MIN(cat_tower / 5,
kapasitas_ally - cat_ally)
                JIKA jumlah_beri > 10 DAN bisa
transfer:
                    transfer cat ke ally
                    BERHENTI

    JIKA aksi tidak siap ATAU cat tower < 220:
SELESAI

    enemyPaintNearby ← jumlah petak cat musuh dalam
jangkauan sensor

    JIKA enemyPaintNearby > 12:
        toSpawn ← MOPPER

    JIKA TIDAK, JIKA isSmallMap:
        JIKA ronde < 15:
            toSpawn ← SOLDIER
        JIKA TIDAK:
            mod ← spawnCounter mod 10
            JIKA mod < 5 → toSpawn ← SPLASHER
            JIKA mod < 8 → toSpawn ← SOLDIER
            SELAIN ITU  → toSpawn ← MOPPER

    JIKA TIDAK, JIKA isMediumMap:
        JIKA ronde < 50:
            toSpawn ← SOLDIER
        JIKA TIDAK:
            mod ← spawnCounter mod 10
            JIKA mod < 4 → toSpawn ← SPLASHER
            JIKA mod < 8 → toSpawn ← SOLDIER
            SELAIN ITU  →
```

Soldier

```
FUNGSI runSoldier(rc):

    JIKA sectorTarget = null:
        id ← Robot ID
        switch (id mod 4):
            0 → sectorTarget ← (3/4 lebar, 3/4
tinggi)
            1 → sectorTarget ← (1/4 lebar, 3/4
tinggi)
            2 → sectorTarget ← (3/4 lebar, 1/4
tinggi)
            3 → sectorTarget ← (1/4 lebar, 1/4
tinggi)

    JIKA tryReloadPaint(rc) = true: SELESAI

    targetRuin   ← null
    bestRuinScore ← -tak hingga
    UNTUK setiap tile yang terlihat:
        JIKA tile bukan ruins ATAU ada robot di
atasnya: lanjut
        distToRuin   ← jarak²(posisi, tile)
        distToCenter ← jarak²(tile, mapCenter)
        distToEnemy  ← jarak²(tile,
predictedEnemyBase) [atau 0 jika null]
        score ← 800 / (distToRuin + 1)
                - distToCenter × 1.2
                - distToEnemy  × 0.3
        JIKA score > bestRuinScore:
            bestRuinScore ← score
            targetRuin ← tile

    JIKA targetRuin ada:
        distToRuin  ← jarak²(posisi, targetRuin)
        typeToBuild ← fpe(rc)

        JIKA distToRuin ≤ 8:
            JIKA pola belum ditandai DAN bisa mark:
                markTowerPattern(typeToBuild,
```

```
targetRuin)

            JIKA canCompleteTowerPattern:
                completeTowerPattern(typeToBuild,
targetRuin)
                sectorTarget ← null
                idleTimer ← 0
                SELESAI

            JIKA aksi siap:
                UNTUK setiap patternTile dalam
radius 8 dari ruins:
                    JIKA penanda tile ≠ cat tile DAN
penanda ≠ EMPTY:
                        attack patternTile dengan
warna sesuai penanda
                        BERHENTI
                JIKA setelah cat bisa complete:
selesaikan tower, SELESAI

            JIKA gerakan siap DAN distToRuin > 2:
                smartMoveTo(rc, targetRuin, false)
            idleTimer ← 0
            SELESAI

        SELAIN ITU:
            JIKA aksi siap:
                UNTUK setiap ruins yang terlihat:
                    enemyNearRuin ← jumlah cat musuh
dalam radius 12
                    JIKA enemyNearRuin > 2 DAN
enemyNearRuin < 10:
                        UNTUK setiap patternTile
dalam radius 8:
                            JIKA patternTile cat
musuh DAN bisa attack:
                                attack patternTile
                                BERHENTI
```

```
            JIKA gerakan siap: smartMoveTo(rc,
targetRuin, false)
            JIKA aksi siap: doSoldierAttack(rc,
posisi)
            SELESAI

    JIKA aksi siap DAN turnCount > 60:
        JIKA posisi.x mod 4 = 2 DAN posisi.y mod 4 =
2:
            nearRuin ← false
            UNTUK setiap ruins di knownRuins:
                JIKA jarak²(posisi, ruins) ≤ 16:
nearRuin ← true; BERHENTI
            JIKA NOT nearRuin DAN bisa
markResourcePattern:
                markResourcePattern(posisi)

    JIKA aksi siap: doSoldierAttack(rc, posisi)

    JIKA gerakan siap:
        idleTimer ← idleTimer + 1
        JIKA idleTimer > IDLE_THRESHOLD:
            JIKA predictedEnemyBase ada:
                sectorTarget ← predictedEnemyBase
            JIKA TIDAK:
                newSector ← (id mod 4 + turnCount /
30) mod 4
                sectorTarget ← kuadran sesuai
newSector
            idleTimer ← 0
        fluidExploreMove(rc, true)
```

Mopper

```
FUNGSI runMopper(rc):

    JIKA cat < 12 DAN tryReloadPaint(rc) = true:
SELESAI
```

```
    acted   ← false
    enemies ← semua robot musuh dalam jangkauan
sensor

    JIKA aksi siap DAN ada musuh:
        bestDir  ← null
        bestHits ← 0
        UNTUK setiap arah d dari 8 arah:
            JIKA tidak bisa mopSwing ke d: lanjut
            hits ← 0
            front      ← posisi + d
            frontLeft  ← posisi + rotasi_kiri(d)
            frontRight ← posisi + rotasi_kanan(d)
            UNTUK setiap musuh:
                JIKA musuh ada di
front/frontLeft/frontRight: hits++
            JIKA hits > bestHits:
                bestHits ← hits
                bestDir  ← d
        JIKA bestDir ada DAN bestHits ≥ 1:
            lakukan mopSwing ke bestDir
            acted ← true

    JIKA aksi siap DAN NOT acted DAN cat mopper >
40:
        bestAlly    ← null
        lowestPaint ← tak hingga
        UNTUK setiap ally dalam radius 2:
            JIKA ally adalah tower: lanjut
            JIKA cat ally < 35% kapasitas DAN cat
ally < lowestPaint:
                lowestPaint ← cat ally
                bestAlly    ← ally
        JIKA bestAlly ada:
            jumlah ← MIN(cat mopper - 15, kapasitas
- cat bestAlly)
            JIKA jumlah > 5 DAN bisa transfer:
transfer cat ke bestAlly
            acted ← true
```

```
    JIKA aksi siap DAN NOT acted:
        bestMop   ← null
        bestScore ← -1
        UNTUK setiap tile cat musuh dalam jangkauan
aksi:
            JIKA tidak bisa attack: lanjut
            dist  ← jarak²(posisi, tile), min 1
            robotDi ← robot di tile tersebut
            score ← 10 / dist
            JIKA ada robot musuh di tile: score ←
score + 25
            JIKA score > bestScore: bestScore ←
score; bestMop ← tile
        JIKA bestMop ada: attack bestMop

    JIKA gerakan siap: fluidExploreMove(rc, false)
```

Splasher

```
FUNGSI runSplasher(rc):

    JIKA tryReloadPaint(rc) = true: SELESAI

    enemyVisible ← jumlah cat musuh yang terlihat
dalam sensor
    JIKA enemyVisible > 4:
        consecutiveEnemyPaintSeen ←
MIN(consecutiveEnemyPaintSeen + 2, 20)
    JIKA TIDAK:
        consecutiveEnemyPaintSeen ←
MAX(consecutiveEnemyPaintSeen - 1, 0)
    reclaimMode ← (consecutiveEnemyPaintSeen ≥ 5)

    JIKA aksi siap:
        bestSplash ← null
        bestScore  ← -tak hingga

        UNTUK setiap kandidat lokasi splash dalam
```

```
jangkauan aksi:
            JIKA tidak bisa attack: lanjut
            paintValue ← 0
            score      ← 0

            UNTUK setiap tile dalam radius AoE
(jarak² ≤ 4) dari kandidat:
                JIKA tile cat musuh:
                    paintValue ← paintValue + 10
                    JIKA reclaimMode: score ← score
+ 15
                    JIKA TIDAK:     score ← score +
7
                JIKA tile kosong DAN passable DAN
bukan ruins:
                    paintValue ← paintValue + 5
                    JIKA reclaimMode: score ← score
+ 3
                    JIKA TIDAK:     score ← score +
9
                JIKA tile cat sekutu ATAU tembok:
                    score ← score - 1.5

            UNTUK setiap ruins di knownRuins:
                JIKA jarak²(kandidat, ruins) ≤ 8 DAN
tidak ada tower di ruins:
                    score ← score + 8

            dist  ← jarak²(posisi, kandidat), min 1
            score ← (score × 7) / dist

            threshold ← 50 jika reclaimMode, 30 jika
ekspansi
            JIKA paintValue ≥ threshold DAN score >
bestScore:
                bestScore  ← score
                bestSplash ← kandidat

        JIKA bestSplash = null:
```

```
            UNTUK setiap kandidat:
                JIKA TIDAK bisa attack: lanjut
                pv ← hitung paintValue (musuh +10,
kosong +5)
                JIKA pv ≥ 20: bestSplash ← kandidat;
BERHENTI

        JIKA bestSplash ada: attack bestSplash

    JIKA gerakan siap:
        idleTimer ← idleTimer + 1
        JIKA idleTimer > IDLE_THRESHOLD:
            sectorTarget ← predictedEnemyBase [atau
mapCenter jika null]
            idleTimer ← 0
        fluidExploreMove(rc, NOT reclaimMode)
```

Fungsi Helper

```
FUNGSI tryReloadPaint(rc) → boolean:
    threshold ← kapasitas × pct_per_tipe

    towerInRange   ← ada tower musuh dalam jangkauan
aksi
    enemyNear      ← ada robot musuh dalam jangkauan
aksi
    finishingTower ← soldier hampir selesai
membangun tower
                     (tile pola yang belum dicat ≤ 3
DAN cat cukup)

    JIKA cat < threshold DAN NOT enemyNear
                         DAN NOT towerInRange
                         DAN NOT finishingTower:
        tower ← findNearestTower(rc)
        JIKA tower ada:
            JIKA jarak²(posisi, tower) ≤ 2:
                tarik cat dari tower sampai penuh
            JIKA TIDAK DAN gerakan siap:
```

```
                smartMoveTo(rc, tower, true)
            return true
    return false


FUNGSI doSoldierAttack(rc, myLoc):
    best      ← null
    bestScore ← -1
    UNTUK setiap tile dalam jangkauan aksi:
        JIKA tidak bisa attack: lanjut
        dist ← jarak²(myLoc, tile), min 1
        JIKA tile adalah tower musuh:
            score ← 3000 / dist
        JIKA TIDAK, JIKA tile cat musuh:
            borderingAlly ← ada petak sekutu
bersebelahan dengan tile ini
            JIKA borderingAlly: score ← 180 / dist
            JIKA TIDAK:         score ← 80  / dist
        JIKA TIDAK, JIKA tile kosong DAN passable:
            score ← 15 / dist
        JIKA score > bestScore: bestScore ← score;
best ← tile
    JIKA best ada DAN bestScore > 0: attack best


FUNGSI smartMoveTo(rc, target, preferAllied):
    bestDir  ← null
    minScore ← tak hingga
    UNTUK setiap arah d dari 8 arah:
        JIKA tidak bisa bergerak ke d: lanjut
        next     ← posisi + d
        score    ← jarak²(next, target) × 10
        tileType ← mapMemory[next]

        JIKA preferAllied DAN tileType = 2 (cat
sekutu): score ← score - 260

        JIKA paintRatio < 0.5:
            JIKA tileType = 3 (cat musuh): score ←
```

```
score + 320
            JIKA tileType = 1 (kosong):    score ←
score + 90

        JIKA dir = lastDir:                 score ←
score - 25
        JIKA dir = rotasi ±1 dari lastDir:  score ←
score - 8

        JIKA next ada di visitedTiles: score ← score
+ 4000

        JIKA score < minScore: minScore ← score;
bestDir ← d

    JIKA bestDir ada: bergerak ke bestDir; lastDir ←
bestDir; recordVisit


FUNGSI fluidExploreMove(rc, useCompassSector):
    bestDir   ← null
    maxScore  ← -tak hingga
    curType   ← mapMemory[posisiSaatIni]
    enemyNear ← ada robot musuh dalam sensor

    UNTUK setiap arah d dari 8 arah:
        JIKA tidak bisa bergerak ke d: lanjut
        next     ← posisi + d
        nextType ← mapMemory[next]
        score    ← 0

        JIKA curType = 2 DAN (nextType = 1 ATAU 3):
score ← score + 750
        JIKA TIDAK, JIKA nextType = 1 ATAU 3:
score ← score + 480
        JIKA TIDAK, JIKA nextType = 0 (unknown):
score ← score + 220
        JIKA TIDAK, JIKA nextType = 2 (cat
sendiri):score ← score - 80
```

```
        JIKA useCompassSector DAN sectorTarget ada:
            JIKA next lebih dekat ke sectorTarget:
score ← score + 420
            JIKA next lebih jauh dari sectorTarget:
score ← score - 200

        JIKA NOT useCompassSector DAN
predictedEnemyBase ada:
            JIKA next lebih dekat ke
predictedEnemyBase: score ← score + 220

        allyNear ← jumlah ally dalam radius 4 dari
next
        JIKA enemyNear: score ← score + allyNear ×
200
        JIKA TIDAK:     score ← score - allyNear ×
140

        JIKA nextType = 2: score ← score + 75

        JIKA d adalah diagonal: score ← score + 45

        JIKA d = lastDir:                  score ←
score + 100
        JIKA d = rotasi ±1 dari lastDir:   score ←
score + 40

        JIKA paintRatio < 0.65:
            distHome ← jarak² ke tower sekutu
terdekat
            mult ← 1 jika isSmallMap, 3 jika
isMediumMap, 5 jika besar
            score ← score - distHome × mult

        JIKA paintRatio < 0.45 DAN nextType = 3:
score ← score - 320

        JIKA next ada di visitedTiles: score ← score
```

```
- 2200

        score ← score + acak(0..54)

        JIKA score > maxScore: maxScore ← score;
bestDir ← d

    JIKA bestDir ada: bergerak ke bestDir; lastDir ←
bestDir; recordVisit


FUNGSI detectSymmetry(rc):
    JIKA symmetryType sudah diketahui DAN
predictedEnemyBase sudah dihitung: SELESAI

    canBeRotational ← true
    canBeHorizontal ← true
    canBeVertical   ← true
    UNTUK setiap tile tembok/ruin yang terlihat:
        rotPair ← (W-1-x, H-1-y)
        hPair   ← (x, H-1-y)
        vPair   ← (W-1-x, y)
        JIKA bisa lihat rotPair DAN tipe berbeda:
canBeRotational ← false
        JIKA bisa lihat hPair   DAN tipe berbeda:
canBeHorizontal ← false
        JIKA bisa lihat vPair   DAN tipe berbeda:
canBeVertical   ← false

    JIKA canBeRotational: symmetryType ← 1
    JIKA TIDAK, JIKA canBeHorizontal: symmetryType ←
2
    JIKA TIDAK, JIKA canBeVertical:   symmetryType ←
3
    SELAIN ITU: symmetryType ← 1

    JIKA symmetryType diketahui DAN towerCount > 0:
        myTower ← alliedTowers[0]
        switch symmetryType:
```

```
            1 → predictedEnemyBase ← (W-1-x, H-1-y)
            2 → predictedEnemyBase ← (x, H-1-y)
            3 → predictedEnemyBase ← (W-1-x, y)


FUNGSI chooseTowerType(rc):
    n ← jumlah tower saat ini
    JIKA n < 3:        return PAINT_TOWER
    JIKA n mod 3 = 0:  return MONEY_TOWER
    SELAIN ITU:        return PAINT_TOWER


FUNGSI recordVisit(loc):
    visitedTiles[visitedIndex] ← loc
    visitedIndex ← (visitedIndex + 1) mod 60


FUNGSI findNearestTower(rc):
    nearest ← null
    minDist ← tak hingga
    UNTUK setiap tower di alliedTowers:
        dist ← jarak²(posisi, tower)
        JIKA dist < minDist: minDist ← dist; nearest
← tower
    return nearest
```

4.1.1.2 Struktur Data

Berikut struktur data yang digunakan pada kode program Main Bot:

- mapMemory (array 2D integer, 61×61) Array dua dimensi yang menyimpan kondisi setiap petak peta berdasarkan koordinat (x, y). Nilai elemen merepresentasikan kondisi petak: 0 untuk petak yang belum pernah dilihat robot, 1 untuk petak kosong tanpa cat, 2 untuk petak yang sudah diwarnai tim sekutu, 3 untuk petak yang diwarnai musuh, dan 4 untuk tembok atau ruin. Ukuran 61×61 dipilih karena ukuran maksimal peta dalam Battlecode 2025 adalah 60×60, sehingga indeks koordinat (0 hingga 60) dapat ditampung dengan aman. Struktur ini memungkinkan robot mengingat kondisi

peta dari ronde-ronde sebelumnya meskipun petak tersebut sudah berada di luar jangkauan pandang robot saat ini.

- visitedTiles (array MapLocation, ukuran 60) + visitedIndex (integer) *Circular buffer* yang menyimpan 60 koordinat petak terakhir yang diinjak robot. visitedIndex berperan sebagai pointer yang menunjuk posisi mana di array yang akan ditimpa berikutnya. Ketika index mencapai 59, nilainya kembali ke 0 sehingga petak yang paling lama secara otomatis tertimpa oleh petak terbaru tanpa perlu menggeser seluruh isi array. Struktur ini digunakan sebagai mekanisme anti-loop pada fungsi fluidExploreMove dan smartMoveTo, di mana petak yang ada di dalam buffer akan mendapat penalti skor sehingga robot cenderung menghindari lokasi yang baru saja dikunjungi.
- lastDir (Direction) Variabel tunggal yang menyimpan arah langkah terakhir yang diambil robot. Digunakan oleh heuristik *momentum* pada fungsi movement untuk memberikan bonus skor kepada arah yang sama atau berdekat dengan arah sebelumnya, sehingga robot cenderung meluncur lurus melewati rintangan dibanding berbelok-belok setiap giliran.
- alliedTowers (array MapLocation, ukuran 30) + towerCount (integer) Array yang menyimpan koordinat tower sekutu yang pernah dilihat robot. towerCount berfungsi sebagai batas atas iterasi sekaligus penunjuk slot kosong berikutnya. Digunakan oleh fungsi tryReloadPaint untuk menentukan tower terdekat sebagai tujuan pengisian ulang cat, dan oleh fungsi movement sebagai titik referensi heuristik *Leash* agar robot tidak terlalu jauh dari sumber cat saat cadangannya mulai menipis. Tower yang hancur dihapus dari array menggunakan teknik *swap-and-pop* agar daftar selalu akurat.
- spawnCounter (integer) Counter bilangan bulat yang bertambah satu setiap kali tower berhasil men-*spawn* satu unit. Digunakan

pada logika *Map-Aware Spawning* di runTower untuk menentukan tipe unit berikutnya berdasarkan hasil spawnCounter mod 10, sehingga rasio komposisi pasukan (Splasher/Soldier/Mopper) terjaga konsisten sesuai fase permainan tanpa memerlukan struktur data tambahan.

- mapWidth, mapHeight, mapArea (integer) + isSmallMap, isMediumMap (boolean) + mapCenter (MapLocation) Sekumpulan variabel skalar yang diinisialisasi satu kali saat robot pertama kali *spawn*. mapArea digunakan untuk mengklasifikasikan ukuran peta menjadi tiga kategori: kecil (< 900 tile), sedang (900–2500 tile), dan besar (> 2500 tile), yang masing-masing tersimpan dalam flag boolean isSmallMap dan isMediumMap. Klasifikasi ini memengaruhi ambang batas fase spawning dan multiplier *Leash*. mapCenter menyimpan koordinat titik tengah peta dan digunakan sebagai faktor penilaian pada heuristik *Forward Operating Base* agar soldier memprioritaskan ruins yang letaknya lebih ke tengah peta.
- symmetryType (integer) + predictedEnemyBase (MapLocation) symmetryType menyimpan hasil deteksi simetri peta dengan nilai: 0 = belum diketahui, 1 = rotasional, 2 = horizontal, 3 = vertikal. Deteksi dilakukan secara inkremental setiap ronde dengan memeriksa konsistensi posisi tembok dan ruin terhadap ketiga kemungkinan simetri. Setelah simetri terdeteksi, predictedEnemyBase dihitung satu kali sebagai prediksi lokasi base musuh berdasarkan posisi tower sekutu pertama yang diketahui. Kedua variabel ini digunakan oleh fungsi fluidExploreMove untuk mengarahkan Mopper dan Splasher ke sisi peta musuh, serta oleh heuristik *Idle Timer Redirect* pada Soldier ketika robot terlalu lama tidak menemukan objektif baru.
- sectorTarget (MapLocation) Variabel yang menyimpan koordinat target kuadran eksplorasi milik satu robot, dihitung satu kali saat

robot pertama kali dijalankan berdasarkan robotID mod 4. Keempat kemungkinan nilai menghasilkan target di empat pojok peta (NE, NW, SE, SW), memastikan robot-robot yang berbeda ID secara otomatis tersebar ke penjuru peta tanpa komunikasi eksplisit. Nilai ini dapat diperbarui oleh mekanisme *Idle Timer Redirect* atau setelah robot berhasil menyelesaikan pembangunan tower.

- idleTimer (integer) + IDLE_THRESHOLD (integer konstan = 30) idleTimer adalah counter yang bertambah setiap giliran selama robot bergerak tanpa menemukan objektif baru (tidak membangun tower, tidak dalam mode reload). Ketika nilainya melebihi IDLE_THRESHOLD, robot mereset sectorTarget ke prediksi base musuh atau ke kuadran baru yang dirotasi berdasarkan turnCount, lalu idleTimer dikembalikan ke 0. Mekanisme ini mencegah robot terjebak berputar-putar di area yang sudah sepenuhnya diwarnai, terutama pada peta dengan banyak tembok seperti *CastleDefense*.
- consecutiveEnemyPaintSeen (integer) Counter yang melacak seberapa konsisten splasher melihat cat musuh di sekitarnya. Nilainya bertambah 2 setiap giliran splasher melihat lebih dari 4 tile cat musuh, dan berkurang 1 setiap giliran tidak melihatnya, dengan batas maksimal 20. Ketika nilai mencapai 5 atau lebih, splasher beralih ke *reclaim mode* yang memprioritaskan overwrite cat musuh dibanding mewarnai tile kosong. Penggunaan counter akumulatif (bukan pengecekan instan) bertujuan mencegah splasher berganti mode secara fluktuatif setiap giliran akibat pergerakan robot.
- knownRuins (array MapLocation, ukuran 50) + ruinCount (integer) Array yang menyimpan koordinat semua ruins yang pernah dilihat robot sejak pertama kali di-*spawn*. Digunakan oleh heuristik *SRP Detection* pada Soldier untuk menghindari penandaan Special Resource Pattern terlalu dekat dengan ruins

(yang akan mengganggu pola tower), serta oleh Splasher untuk memberikan bonus skor pada titik splash yang berdekatan dengan ruins yang belum memiliki tower, karena splash di area tersebut sekaligus dapat menghambat musuh membangun tower di ruins tersebut.

4.1.2 Bot Alternatif 1: Argha

4.1.2.1 Pseudocode

Program Utama

```
PROGRAM RobotPlayer:

VARIABEL GLOBAL:
    spawnCounter    ← 0
    visitedTiles    ← array[30]
    visitedIndex    ← 0
    knownTowers     ← array[30]
    knownTowerCount ← 0

FUNGSI run(rc):
    ULANGI SELAMANYA:
        rememberTowers(rc)

        JIKA tipe robot = SOLDIER  → jalankan
runSoldier(rc)
        JIKA tipe robot = MOPPER   → jalankan
runMopper(rc)
        JIKA tipe robot = SPLASHER → jalankan
runSplasher(rc)
        SELAIN ITU (tower)         → jalankan
runTower(rc)

        tunggu giliran berikutnya
```

Tower

```
FUNGSI runTower(rc):
```

```
    JIKA bisa upgrade tower saat ini:
        upgrade tower

    enemies ← semua robot musuh dalam jangkauan
sensor
    UNTUK setiap musuh:
        JIKA bisa serang musuh:
            serang musuh
            BERHENTI

    allies ← semua robot sekutu dalam jangkauan
sensor
    UNTUK setiap ally (Soldier/Mopper/Splasher):
        JIKA cat ally < 1/3 kapasitas maksimalnya:
            jumlah_beri ← MIN(cat_tower / 4,
kapasitas_ally - cat_ally)
            JIKA jumlah_beri > 5 DAN bisa transfer:
                transfer cat ke ally
                BERHENTI

    enemyPaintCount ← jumlah petak cat musuh di
sekitar tower

    JIKA cat tower < 200:
        SELESAI

    JIKA enemyPaintCount > 8:
        toSpawn ← MOPPER

    JIKA TIDAK, JIKA ronde < 100:
        toSpawn ← SOLDIER

    JIKA TIDAK, JIKA ronde < 300:
        r ← spawnCounter mod 10
        JIKA r < 4  → toSpawn ← SPLASHER
        JIKA r < 9  → toSpawn ← SOLDIER
        SELAIN ITU  → toSpawn ← MOPPER

    SELAIN ITU:
```

```
        r ← spawnCounter mod 10
        JIKA r < 6  → toSpawn ← SPLASHER
        JIKA r < 8  → toSpawn ← SOLDIER
        SELAIN ITU  → toSpawn ← MOPPER

    bestSpawnDir  ← null
    bestSpawnSkor ← -tak hingga

    UNTUK setiap arah d dari 8 arah:
        spawnLoc ← posisi tower + d
        JIKA tidak bisa bangun robot di spawnLoc:
lanjut

        skor ← 0
        UNTUK setiap petak t dalam jangkauan sensor:
            JIKA jarak(spawnLoc, t) ≤ 4 DAN t adalah
cat musuh:
                skor ← skor + 1
        skor ← skor - jumlah robot sekutu di sekitar
spawnLoc

        JIKA skor > bestSpawnSkor:
            bestSpawnSkor ← skor
            bestSpawnDir  ← d

    JIKA bestSpawnDir tidak null:
        bangun toSpawn di posisi(tower +
bestSpawnDir)
        spawnCounter ← spawnCounter + 1
```

Soldier

```
FUNGSI runSoldier(rc):
    myLoc    ← posisi saat ini
    curPaint ← cat saat ini
    maxPaint ← kapasitas cat maksimal

    nearestRuin     ← null
    nearestRuinDist ← tak hingga
```

```
    UNTUK setiap petak tile yang terlihat:
        JIKA tile adalah ruins DAN tidak ada robot
di atasnya:
            dist ← jarak(myLoc, tile)
            JIKA dist ≤ 4 DAN dist <
nearestRuinDist:
                nearestRuinDist ← dist
                nearestRuin     ← tile

    paintRatio   ← curPaint / maxPaint
    nearestTower ← findNearestKnownTower(rc)

    JIKA paintRatio < 0.30 DAN nearestTower ada:
        JIKA jarak(myLoc, nearestTower) ≤ 2:
            tarik cat dari tower sampai penuh
        JIKA TIDAK:
            navigateToward(rc, nearestTower)
            warnai petak di bawah kaki
            SELESAI

    JIKA nearestRuin ada:
        tt ← chooseTowerType(rc)

        JIKA nearestRuinDist ≤ 2:
            tandai pola tower (markTowerPattern)
            JIKA bisa selesaikan tower sekarang:
                selesaikan tower
            JIKA TIDAK:
                warnai petak-petak yang ditandai
sesuai pola
                JIKA sekarang bisa diselesaikan:
selesaikan tower
            SELESAI

        JIKA TIDAK:
            navigateToward(rc, nearestRuin)
            warnai petak di bawah kaki
            SELESAI
```

```
    JIKA aksi siap:
        JIKA petak di bawah kaki bukan cat sekutu:
            warnai petak di bawah kaki
        JIKA TIDAK:
            bestPaint      ← null
            bestPaintScore ← 0

            UNTUK setiap petak tile yang bisa
diserang:
                JIKA cat tile adalah MUSUH:
                    JIKA ada petak sekutu
bersebelahan dengan tile ini:
                        s ← 5
                    JIKA TIDAK:
                        s ← 2
                JIKA cat tile KOSONG DAN passable
DAN bukan ruins:
                    s ← 1
                SELAIN ITU:
                    s ← 0

                JIKA s > bestPaintScore:
                    bestPaintScore ← s
                    bestPaint      ← tile

            JIKA bestPaint ada: warnai bestPaint

    JIKA gerakan siap:
        bestDir   ← null
        bestScore ← -tak hingga

        UNTUK setiap arah d dari 8 arah:
            JIKA tidak bisa bergerak ke d: lanjut
            dest          ← myLoc + d
            traversalCost ← paintCostOf(cat di dest)
            newTiles      ← countUnpaintedAround(rc,
dest)
```

```
            score ← newTiles × 12
            score ← score - traversalCost × 15

            JIKA paintRatio < 0.4: score ← score -
traversalCost × 10
            JIKA paintRatio < 0.2 DAN traversalCost
> 0: score ← score - 30

            score ← score - 8 × jumlah robot sekutu
di sekitar dest
            score ← score - visitedPenalty(dest)
            score ← score + acak(0..2)

            JIKA score > bestScore:
                bestScore ← score
                bestDir   ← d

        JIKA bestDir ada:
            bergerak ke bestDir
            recordVisit(posisi baru)
        JIKA TIDAK:
            bergerak ke arah acak (darurat)
            recordVisit(posisi baru)
```

Mopper

```
FUNGSI runMopper(rc):
    myLoc    ← posisi saat ini
    curPaint ← cat saat ini

    JIKA curPaint < 20:
        tower ← findNearestKnownTower(rc)
        JIKA tower ada:
            JIKA jarak(myLoc, tower) ≤ 2:
                tarik cat dari tower
            JIKA TIDAK:
                navigateToward(rc, tower)
                SELESAI
```

```
    UNTUK setiap ally Soldier dalam jangkauan
sensor:
        JIKA cat soldier < 100 DAN jarak ke soldier
≤ 2:
            transfer ← MIN(curPaint - 10, 200 -
cat_soldier)
            JIKA transfer > 0: transfer cat ke
soldier
            BERHENTI

    enemies  ← semua robot musuh dalam jangkauan
sensor
    didSwing ← false

    UNTUK setiap arah d dari 8 arah:
        JIKA bisa mop swing ke d:
            JIKA countEnemiesInSwingDir(d, enemies)
≥ 2:
                lakukan mop swing ke d
                didSwing ← true
                BERHENTI

    JIKA gerakan siap:
        bestDir   ← null
        bestScore ← -tak hingga

        UNTUK setiap arah d dari 8 arah:
            JIKA tidak bisa bergerak ke d: lanjut
            dest          ← myLoc + d
            traversalCost ← paintCostOf(cat di dest)

            score ← 0
            score ← score - traversalCost × 8
            JIKA dest cat musuh  : score ← score +
20
            JIKA dest cat sekutu : score ← score -
15

            UNTUK setiap musuh:
```

```
                JIKA jarak(dest, musuh) ≤ 4: score ←
score + 25

            further ← dest + d
            JIKA bisa lihat further DAN further cat
musuh:
                score ← score + 10

            score ← score - visitedPenalty(dest)
            score ← score + acak(0..2)

            JIKA score > bestScore:
                bestScore ← score
                bestDir   ← d

        bergerak ke bestDir (atau acak jika null)
        recordVisit(posisi baru)

    JIKA NOT didSwing:
        UNTUK setiap arah d dari 8 arah:
            JIKA bisa mop swing ke d:
                JIKA countEnemiesInSwingDir(d,
enemies) ≥ 1:
                    lakukan mop swing ke d
                    didSwing ← true
                    BERHENTI

    JIKA NOT didSwing:
        UNTUK setiap arah d dari 8 arah:
            target ← myLoc + d
            JIKA target adalah cat musuh DAN bisa
serang:
                serang target
                BERHENTI
```

Splasher

```
FUNGSI runSplasher(rc):
    myLoc      ← posisi saat ini
```

```
    curPaint   ← cat saat ini
    maxPaint   ← kapasitas cat maksimal
    paintRatio ← curPaint / maxPaint

    JIKA paintRatio < 0.25:
        tower ← findNearestKnownTower(rc)
        JIKA tower ada:
            JIKA jarak(myLoc, tower) ≤ 2:
                tarik cat dari tower penuh
            JIKA TIDAK DAN gerakan siap:
                navigateToward(rc, tower)
                SELESAI

    JIKA gerakan siap:
        bestDir       ← null
        bestMoveScore ← -tak hingga

        UNTUK setiap arah d dari 8 arah:
            JIKA tidak bisa bergerak ke d: lanjut
            dest          ← myLoc + d
            traversalCost ← paintCostOf(cat di dest)

            score ← 0
            score ← score - traversalCost × 12

            enemyAround ← countEnemyPaintAround(rc,
dest)
            score ← score + enemyAround × 15

            JIKA dest cat musuh : score ← score + 10
            JIKA dest kosong    : score ← score + 5

            JIKA paintRatio < 0.3: score ← score -
traversalCost × 10

            score ← score - visitedPenalty(dest)
            score ← score + acak(0..2)

            JIKA score > bestMoveScore:
```

```
                bestMoveScore ← score
                bestDir       ← d

        bergerak ke bestDir (atau acak jika null)
        recordVisit(posisi baru)

    JIKA aksi siap:
        bestSplash      ← null
        bestSplashScore ← 0

        UNTUK setiap kandidat lokasi splash dalam
jangkauan aksi:
            JIKA tidak bisa serang lokasi ini:
lanjut

            enemyPaint ← 0
            empty      ← 0
            allyPaint  ← 0

            UNTUK setiap petak t di sekitar kandidat
(jarak ≤ 4):
                JIKA t cat musuh  : enemyPaint ←
enemyPaint + 1
                JIKA t kosong     : empty      ←
empty + 1
                JIKA t cat sekutu : allyPaint  ←
allyPaint + 1

            score ← 4×enemyPaint + 2×empty -
2×allyPaint

            JIKA score > bestSplashScore:
                bestSplashScore ← score
                bestSplash      ← kandidat

        JIKA bestSplash ada: lakukan splash di
bestSplash
```

Fungsi Helper

```
FUNGSI paintCostOf(tipe_cat):
    JIKA cat sekutu : return 0
    JIKA cat musuh  : return 2
    SELAIN ITU      : return 1

FUNGSI countUnpaintedAround(rc, loc):
    count ← 0
    UNTUK setiap arah d dari 8 arah:
        adj ← loc + d
        JIKA bisa lihat adj DAN adj passable DAN
bukan cat sekutu:
            count ← count + 1
    return count

FUNGSI countEnemyPaintAround(rc, loc):
    count ← 0
    UNTUK setiap arah d dari 8 arah:
        adj ← loc + d
        JIKA bisa lihat adj DAN adj adalah cat
musuh:
            count ← count + 1
    return count

FUNGSI recordVisit(loc):
    visitedTiles[visitedIndex] ← loc
    visitedIndex ← (visitedIndex + 1) mod 30

FUNGSI visitedPenalty(loc):
    UNTUK setiap lokasi di visitedTiles:
        JIKA lokasi = loc: return 100
    return 0

FUNGSI rememberTowers(rc):
    UNTUK setiap tower di knownTowers:
        JIKA bisa lihat lokasi tower:
            JIKA tidak ada tower sekutu di lokasi
itu:
                hapus dari knownTowers
```

```
    UNTUK setiap ally dalam jangkauan sensor:
        JIKA ally adalah tower (bukan
Soldier/Mopper/Splasher):
            JIKA belum ada di knownTowers:
                tambah ke knownTowers

FUNGSI findNearestKnownTower(rc):
    nearest ← null
    minDist ← tak hingga
    UNTUK setiap tower di knownTowers:
        dist ← distanceSquared(posisi_saat_ini,
tower)
        JIKA dist < minDist:
            minDist ← dist
            nearest ← tower
    return nearest

FUNGSI navigateToward(rc, target):
    bestDir   ← null
    bestScore ← tak hingga

    UNTUK setiap arah d dari 8 arah:
        JIKA tidak bisa bergerak ke d: lanjut
        next  ← posisi + d
        dist  ← distanceSquared(next, target)
        cost  ← paintCostOf(cat di next)
        score ← dist + cost×15 +
visitedPenalty(next)

        JIKA score < bestScore:
            bestScore ← score
            bestDir   ← d

    JIKA bestDir ada:
        bergerak ke bestDir
        recordVisit(posisi baru)

FUNGSI countEnemiesInSwingDir(rc, dir, enemies):
    front      ← posisi + dir
```

```
    frontLeft  ← posisi + rotasi_kiri(dir)
    frontRight ← posisi + rotasi_kanan(dir)
    count ← 0
    UNTUK setiap musuh:
        JIKA musuh ada di front ATAU frontLeft ATAU
frontRight:
            count ← count + 1
    return count

FUNGSI chooseTowerType(rc):
    n ← jumlah tower saat ini
    JIKA n < 3        : return PAINT_TOWER
    JIKA n mod 3 = 0  : return MONEY_TOWER
    SELAIN ITU        : return PAINT_TOWER
```

4.1.2.2 Struktur Data

Berikut struktur data yang digunakan pada kode program Bot Alternatif 1:

- spawnCounter (integer): Counter bilangan bulat yang bertambah satu setiap kali tower berhasil men-spawn satu unit. Digunakan pada logika penentuan tipe unit di runTower untuk menentukan tipe unit berikutnya berdasarkan hasil spawnCounter mod 10, sehingga rasio komposisi pasukan (Splasher/Soldier/Mopper) terjaga konsisten sesuai fase permainan tanpa memerlukan struktur data tambahan.
- visitedTiles (array MapLocation, ukuran 30) + visitedIndex (integer): Circular buffer yang menyimpan 30 koordinat petak terakhir yang diinjak robot. visitedIndex berperan sebagai pointer yang menunjuk posisi mana di array yang akan ditimpa berikutnya. Ketika index mencapai 29, nilainya kembali ke 0 sehingga petak yang paling lama secara otomatis tertimpa oleh petak terbaru tanpa perlu menggeser seluruh isi array. Struktur ini digunakan sebagai mekanisme anti-loop pada fungsi navigateToward dan logika pergerakan di runSoldier, runMopper, serta runSplasher, di mana petak yang ada di dalam buffer akan mendapat penalti skor sebesar

100 sehingga robot cenderung menghindari lokasi yang baru saja dikunjungi.

- knownTowers (array MapLocation, ukuran 30) + knownTowerCount (integer): Array yang menyimpan koordinat tower sekutu yang pernah dilihat robot. knownTowerCount berfungsi sebagai batas atas iterasi sekaligus penunjuk slot kosong berikutnya. Digunakan oleh fungsi findNearestKnownTower untuk menentukan tower terdekat sebagai tujuan pengisian ulang cat pada runSoldier, runMopper, dan runSplasher. Tower yang sudah hancur atau tidak lagi terdeteksi sebagai tower sekutu dihapus dari array menggunakan teknik swap-and-pop agar daftar selalu akurat, dikelola oleh fungsi rememberTowers.

Berikut fungsi dan prosedur yang digunakan pada kode program Bot Alternatif 1:

- run(rc): Fungsi utama yang dijalankan sepanjang hidup robot. Berisi loop tak terbatas yang setiap rondenya memanggil rememberTowers untuk memperbarui memori tower, kemudian mendelegasikan aksi ke fungsi yang sesuai berdasarkan tipe robot menggunakan struktur switch (runSoldier, runMopper, runSplasher, atau runTower).
- rememberTowers(rc): Dipanggil setiap ronde sebelum aksi robot manapun dilakukan. Fungsi ini memiliki dua tanggung jawab: pertama, memeriksa seluruh entri knownTowers dan menghapus lokasi yang robotnya sudah bukan lagi tower sekutu (menggunakan teknik swap-and-pop); kedua, memindai seluruh robot sekutu dalam jangkauan sensor dan menambahkan koordinat tower yang belum tercatat ke dalam knownTowers.
- runTower(rc): Mengelola logika tower dengan urutan prioritas: upgrade diri sendiri, menyerang robot musuh terdekat, mentransfer cat ke robot sekutu yang cadangannya di bawah sepertiga kapasitas, kemudian menentukan tipe unit yang akan di-spawn.

Penentuan tipe spawn bersifat adaptif: spawn Mopper dilakukan secara darurat jika terdeteksi lebih dari 8 petak cat musuh di sekitar tower, sementara di kondisi normal mengikuti tiga fase berbasis ronde (ronde < 100: Soldier penuh; ronde 100–300: Splasher 40%, Soldier 50%, Mopper 10%; ronde ≥ 300: Splasher 60%, Soldier 20%, Mopper 20%). Lokasi spawn dipilih secara greedy berdasarkan skor yang mempertimbangkan jumlah cat musuh di sekitar titik spawn dikurangi jumlah robot sekutu yang sudah ada di area tersebut.

- runSoldier(rc): Mengimplementasikan logika Soldier sebagai Border Guard melalui beberapa prioritas berurutan. Pertama, refill cat ke tower terdekat jika cat di bawah 30% kapasitas. Kedua, mengejar dan menyelesaikan pembangunan tower jika terdapat ruin dengan jarak kuadrat ≤ 4. Ketiga, memilih petak terbaik untuk diwarnai secara greedy berdasarkan skor ancaman (cat musuh di perbatasan wilayah sekutu mendapat skor 5, cat musuh jauh mendapat skor 2, petak kosong mendapat skor 1). Keempat, bergerak ke arah terbaik berdasarkan skor yang memperhitungkan jumlah petak kosong di sekitar tujuan, biaya traversal cat, kepadatan robot sekutu, dan penalti anti-loop.
- runMopper(rc): Mengimplementasikan logika Mopper sebagai Paint Eraser. Mopper secara proaktif mentransfer cat ke Soldier yang cadangannya di bawah 100. Mop swing diprioritaskan ke arah yang mengenai minimal 2 robot musuh sekaligus sebelum bergerak (hemat cooldown), dan setelah bergerak diturunkan threshold-nya menjadi minimal 1 robot musuh. Jika tidak ada swing yang dilakukan, Mopper menyerang petak cat musuh terdekat secara langsung. Pergerakan Mopper dipandu skor yang mengutamakan petak cat musuh (+20), kedekatan dengan robot musuh (+25 per musuh dalam radius 4), dan lookahead satu petak ke depan (+10).

- runSplasher(rc): Mengimplementasikan logika Splasher sebagai Area Reclaimer. Splasher refill cat jika di bawah 25% kapasitas. Pergerakan diarahkan secara greedy ke area dengan cat musuh terbanyak di sekitarnya (bonus +15 per petak cat musuh di sekitar tujuan). Titik pusat serangan AoE dipilih berdasarkan skor 4 × cat_musuh + 2 × petak_kosong - 2 × cat_sekutu untuk memaksimalkan coverage sekaligus meminimalkan friendly fire.
- paintCostOf(p): Fungsi helper yang mengembalikan biaya traversal numerik berdasarkan tipe cat pada suatu petak: 0 untuk cat sekutu, 1 untuk petak kosong, dan 2 untuk cat musuh. Digunakan oleh fungsi pergerakan semua tipe robot untuk memperhitungkan penalti melintas di atas cat musuh.
- countUnpaintedAround(rc, loc): Menghitung jumlah petak passable di delapan arah sekitar loc yang belum diwarnai cat sekutu. Digunakan oleh heuristik pergerakan Soldier untuk mengukur potensi ekspansi wilayah dari suatu posisi tujuan.
- countEnemyPaintAround(rc, loc): Menghitung jumlah petak di delapan arah sekitar loc yang berwarna cat musuh. Digunakan oleh heuristik pergerakan Splasher untuk mengarahkannya ke area konsentrasi cat musuh tertinggi.
- recordVisit(loc): Prosedur yang mencatat koordinat loc ke dalam visitedTiles pada posisi yang ditunjuk visitedIndex, lalu memajukan pointer secara melingkar (mod 30). Dipanggil setiap kali robot berpindah posisi.
- visitedPenalty(loc): Fungsi yang mengembalikan 100 jika loc ditemukan di dalam visitedTiles, dan 0 jika tidak. Digunakan sebagai komponen penalti dalam skor pergerakan semua tipe robot untuk mencegah robot berputar-putar di lokasi yang sama.
- findNearestKnownTower(rc): Fungsi yang mengembalikan koordinat tower sekutu terdekat dari posisi robot saat ini berdasarkan daftar knownTowers. Mengembalikan null jika daftar

kosong. Digunakan oleh runSoldier, runMopper, dan runSplasher sebagai titik tujuan pengisian ulang cat.

- navigateToward(rc, target): Fungsi navigasi yang memilih arah terbaik menuju target dengan meminimasi skor gabungan dari jarak kuadrat ke target, biaya cat petak tujuan (dikali 15), dan penalti anti-loop dari visitedPenalty. Setelah bergerak, posisi baru dicatat via recordVisit.
- paintCurrentTile(rc): Prosedur yang mewarnai petak di posisi robot saat ini jika belum berwarna cat sekutu. Dipanggil oleh Soldier saat berjalan menuju tower untuk refill, agar setiap langkah perjalanan tetap meninggalkan jejak cat.
- countEnemiesInSwingDir(rc, dir, enemies): Fungsi yang menghitung jumlah robot musuh yang berada di tiga petak area swing (depan, depan-kiri, depan-kanan) untuk arah dir. Digunakan oleh runMopper untuk menentukan apakah mop swing ke suatu arah layak dilakukan berdasarkan threshold jumlah musuh yang akan terkena.
- chooseTowerType(rc): Fungsi yang menentukan tipe tower yang akan dibangun berdasarkan jumlah tower sekutu yang sudah ada. Mengembalikan LEVEL_ONE_MONEY_TOWER jika jumlah tower habis dibagi 3, dan LEVEL_ONE_PAINT_TOWER untuk kondisi lainnya, dengan LEVEL_ONE_PAINT_TOWER selalu dipilih jika jumlah tower masih di bawah 3.
- updateEnemyRobots(rc): Prosedur yang dipanggil oleh runMopper di akhir setiap giliran. Mendeteksi robot musuh di sekitar Mopper dan secara berkala (setiap 20 ronde) mengirimkan jumlah musuh yang terdeteksi sebagai pesan ke seluruh robot sekutu dalam jangkauan, sebagai mekanisme komunikasi antar-unit sederhana.

4.1.3 Bot Alternatif 2: Aurel

4.1.3.1 Pseudocode

Program Utama

```
PROGRAM RobotPlayer:

VARIABEL GLOBAL:
    turnCount             ← 0
    randomNumberGenerator ← Random(1234)
    directions            ← [NORTH, NORTHEAST, EAST,
SOUTHEAST,
                              SOUTH, SOUTHWEST,
WEST, NORTHWEST]
    mapMemory             ← array[61][61]
    visitedTiles          ← array[40]
    visitedIndex          ← 0
    alliedTowers          ← array[30]
    towerCount            ← 0
    knownRuins            ← array[50]
    ruinCount             ← 0

FUNGSI run(rc):
    ULANGI SELAMANYA:
        updateMapMemory(rc)
        updateTowerMemory(rc)
        updateRuinMemory(rc)

        JIKA tipe robot = SOLDIER  → jalankan
runSoldier(rc)
        JIKA tipe robot = SPLASHER → jalankan
runSplasher(rc)
        JIKA tipe robot = MOPPER   → jalankan
runMopper(rc)
        SELAIN ITU (tower)         → jalankan
runTower(rc)

        tunggu giliran berikutnya (Clock.yield)
```

Memory Management

```
FUNGSI updateMapMemory(rc):
    UNTUK setiap tile yang terlihat dalam jarak
pandang (~4.47 petak):
```

```
        loc ← koordinat (x, y) tile tersebut
        JIKA koordinat di luar batas 0-60: lanjut ke
tile berikutnya
        JIKA tile adalah tembok atau ruin  :
mapMemory[x][y] ← 4
        JIKA TIDAK, JIKA tile cat sekutu   :
mapMemory[x][y] ← 2
        JIKA TIDAK, JIKA tile cat musuh    :
mapMemory[x][y] ← 3
        SELAIN ITU (tile kosong)           :
mapMemory[x][y] ← 1

FUNGSI updateTowerMemory(rc):
    UNTUK setiap ally dalam jarak pandang maksimal:
        JIKA ally bukan tipe tower: lanjut
        known ← false
        UNTUK setiap tower di alliedTowers:
            JIKA lokasi tower = lokasi ally:
                known ← true
                BERHENTI
        JIKA NOT known DAN list belum penuh:
            tambah lokasi ally ke
alliedTowers[towerCount]
            towerCount ← towerCount + 1

FUNGSI updateRuinMemory(rc):
    UNTUK setiap tile yang terlihat:
        JIKA tile bukan ruin: lanjut
        loc ← koordinat tile
        JIKA tile dalam jarak pandang:
            robotAtRuin ← robot di lokasi tersebut
        SELAIN ITU:
            robotAtRuin ← null
        alreadyTower ← (robotAtRuin != null DAN
tipenya tower)

        JIKA alreadyTower:
            UNTUK setiap ruin di knownRuins:
                JIKA lokasi ruin = loc:
```

```
                    timpa ruin ini dengan ruin
terakhir di list
                    ruinCount ← ruinCount - 1
                    BERHENTI
        JIKA TIDAK:
            JIKA loc belum ada di knownRuins dan
list belum penuh:
                tambah loc ke knownRuins[ruinCount]
                ruinCount ← ruinCount + 1
```

Reload Paint

```
FUNGSI tryReloadPaint(rc) → boolean:
    threshold ← 30% dari kapasitas cat maksimal
robot ini
    JIKA cat robot >= threshold: return false

    nearest ← null
    minDist ← INTEGER.MAX_VALUE
    UNTUK setiap tower di alliedTowers:
        distance ← jarak kuadrat (posisi robot,
tower)
        JIKA distance < minDist:
            minDist ← distance
            nearest ← tower

    JIKA nearest = null: return false

    JIKA minDist <= 2:
        JIKA action siap:
            need ← kapasitas maks - cat sekarang
            JIKA need > 0 DAN bisa transfer: tarik
cat dari tower sebanyak need
    JIKA TIDAK DAN gerakan siap:
        moveToTarget(rc, nearest)
    return true
```

Tower Logic

```
FUNGSI runTower(rc):

    JIKA action siap DAN cat >= 300 DAN chips >=
500:
        randomRoll ← acak(0..9)
        JIKA randomRoll < 4 : unit ← SOLDIER
        JIKA randomRoll < 7 : unit ← SPLASHER
        SELAIN ITU          : unit ← MOPPER

        JIKA unit = SPLASHER DAN chips < 400: unit ←
SOLDIER
        JIKA unit = MOPPER   DAN chips < 300: unit ←
SOLDIER

        UNTUK setiap arah d dari 8 arah:
            loc ← posisi tower + d
            JIKA bisa bangun unit di loc:
                bangun unit di loc
                BERHENTI

    UNTUK setiap musuh dalam jangkauan sensor:
        JIKA bisa serang musuh:
            serang musuh
            BERHENTI

    JIKA bisa upgrade tower ini:
        upgrade tower
```

Soldier Logic

```
FUNGSI runSoldier(rc):

    JIKA tryReloadPaint(rc) = true: SELESAI

    myLocation ← posisi robot sekarang
    targetRuin ← null
    UNTUK setiap tile yang terlihat:
        JIKA tile bukan ruin: lanjut
        location ← koordinat tile
```

```
        JIKA tidak bisa sense lokasi itu: lanjut
        robotAtLocation ← robot di lokasi tersebut
        JIKA robotAtLocation = null:
            targetRuin ← tile
            BERHENTI

    JIKA targetRuin ada:
        buildTowerAtRuin(rc, koordinat targetRuin)
        SELESAI

    bestRuin              ← null
    maxDistanceFromTowers ← -1

    UNTUK setiap ruin di knownRuins:
        JIKA ruin = null: lanjut
        minDistanceToTower ← INTEGER.MAX_VALUE
        UNTUK setiap tower di alliedTowers:
            distance ← jarak kuadrat (ruin, tower)
            JIKA distance < minDistanceToTower:
                minDistanceToTower ← distance
        JIKA minDistanceToTower >
maxDistanceFromTowers:
            maxDistanceFromTowers ←
minDistanceToTower
            bestRuin ← ruin

    JIKA bestRuin ada:
        JIKA gerakan siap: moveToTarget(rc,
bestRuin)
        JIKA jarak(myLocation, bestRuin) <= 8:
            JIKA bisa sense bestRuin:
                ruinInfo ← info tile bestRuin
                JIKA ruinInfo ada:
buildTowerAtRuin(rc, bestRuin)
        JIKA action siap: paintCurrentTile(rc)
        SELESAI

    JIKA action siap  : paintCurrentTile(rc)
    JIKA gerakan siap : exploreMap(rc)
```

Soldier Helpers

```
FUNGSI buildTowerAtRuin(rc, ruinLocation):
    towerType ← LEVEL_ONE_PAINT_TOWER

    JIKA bisa completeTowerPattern(towerType,
ruinLocation):
        completeTowerPattern(towerType,
ruinLocation)
        SELESAI

    JIKA bisa markTowerPattern(towerType,
ruinLocation):
        markTowerPattern(towerType, ruinLocation)

    JIKA action siap:
        UNTUK setiap patternTile dalam radius 8 dari
ruinLocation:
            JIKA tile sudah ditandai DAN belum
diwarnai sesuai penanda:
                useSecondaryColor ← (penanda tile =
ALLY_SECONDARY)
                JIKA bisa attack tile:
                    attack tile dengan warna yang
sesuai
                    BERHENTI

    JIKA gerakan siap: moveToTarget(rc,
ruinLocation)


FUNGSI paintCurrentTile(rc):
    currentTile ← info tile di posisi robot sekarang
    JIKA tile bukan cat sekutu DAN bisa attack tile
ini:
        attack tile ini
```

Splasher Logic

```
FUNGSI buildTowerAtRuin(rc, ruinLocation):
    towerType ← LEVEL_ONE_PAINT_TOWER

    JIKA bisa completeTowerPattern(towerType,
ruinLocation):
        completeTowerPattern(towerType,
ruinLocation)
        SELESAI

    JIKA bisa markTowerPattern(towerType,
ruinLocation):
        markTowerPattern(towerType, ruinLocation)

    JIKA action siap:
        UNTUK setiap patternTile dalam radius 8 dari
ruinLocation:
            JIKA tile sudah ditandai DAN belum
diwarnai sesuai penanda:
                useSecondaryColor ← (penanda tile =
ALLY_SECONDARY)
                JIKA bisa attack tile:
                    attack tile dengan warna yang
sesuai
                    BERHENTI

    JIKA gerakan siap: moveToTarget(rc,
ruinLocation)


FUNGSI paintCurrentTile(rc):
    currentTile ← info tile di posisi robot sekarang
    JIKA tile bukan cat sekutu DAN bisa attack tile
ini:
        attack tile ini
```

Splasher Helpers

```
FUNGSI findTowerToSupport(rc) → MapLocation:
    best         ← null
```

```
    maxUnpainted ← 0

    UNTUK setiap tower di alliedTowers:
        JIKA tower di luar jangkauan sensor: lanjut
        unpaintedCount ← 0
        UNTUK setiap tile dalam radius 8 dari tower:
            JIKA tile bukan tembok/ruin DAN bukan
cat sekutu:
                unpaintedCount ← unpaintedCount + 1
        JIKA unpaintedCount > maxUnpainted:
            maxUnpainted ← unpaintedCount
            best ← tower

    return best
```

Mopper Logic

```
FUNGSI runMopper(rc):

    JIKA cat < 10 DAN tryReloadPaint(rc) = true:
SELESAI

    acted ← false

    JIKA action siap:
        UNTUK setiap arah d dari 8 arah:
            JIKA bisa mopSwing ke d:
                lakukan mopSwing ke d
                acted ← true
                BERHENTI

    JIKA action siap DAN NOT acted DAN cat mopper >
50:
        UNTUK setiap ally dalam radius 2:
            JIKA cat ally < 20% kapasitas:
                JIKA bisa transferPaint ke ally:
                    transfer 20 cat ke ally
                    acted ← true
                    BERHENTI
```

```
    JIKA action siap DAN NOT acted:
        bestTarget ← null
        maxScore   ← 0
        UNTUK setiap tile dalam jangkauan aksi
mopper:
            JIKA tile bukan cat musuh ATAU tidak
bisa attack: lanjut
            score ← 1
            UNTUK setiap tile tetangga (radius 2):
                JIKA tetangga cat sekutu: score ←
score + 2
            JIKA score > maxScore:
                maxScore   ← score
                bestTarget ← tile
        JIKA bestTarget ada: attack bestTarget

    JIKA gerakan siap: patrolAllyTerritory(rc)
```

Mopper Helpers

```
FUNGSI patrolAllyTerritory(rc):
    bestDirection ← null
    maxScore      ← -99999

    UNTUK setiap arah d dari 8 arah:
        JIKA tidak bisa bergerak ke d: lanjut
        nextLocation ← posisi + d
        score ← 0

        tileType ←
mapMemory[nextLocation.x][nextLocation.y]
        JIKA tileType = 3 (cat musuh)  : score ←
score + 50
        JIKA tileType = 2 (cat sekutu) : score ←
score + 30
        JIKA tileType = 1 (kosong)     : score ←
score + 10
```

```
        UNTUK setiap tile di visitedTiles:
            JIKA tile = nextLocation: score ← score
- 100

        score ← score + acak(0..4)
        JIKA score > maxScore:
            maxScore      ← score
            bestDirection ← d

    JIKA bestDirection ada:
        bergerak ke bestDirection
        visitedTiles[visitedIndex] ← posisi baru
        visitedIndex ← (visitedIndex + 1) mod 40
```

Movement Helpers

```
FUNGSI moveToTarget(rc, target):
    bestDirection ← null
    minScore      ← INTEGER.MAX_VALUE

    UNTUK setiap arah d dari 8 arah:
        JIKA tidak bisa bergerak ke d: lanjut
        nextLocation ← posisi + d
        distance ← jarak kuadrat (nextLocation,
target)
        penalty  ← 0

        UNTUK setiap tile di visitedTiles:
            JIKA tile = nextLocation: penalty ←
penalty + 1000

        JIKA distance + penalty < minScore:
            minScore      ← distance + penalty
            bestDirection ← d

    JIKA bestDirection ada:
        bergerak ke bestDirection
        visitedTiles[visitedIndex] ← posisi baru
        visitedIndex ← (visitedIndex + 1) mod 40
```

```
FUNGSI exploreMap(rc):
    bestDirection ← null
    maxScore      ← -99999

    UNTUK setiap arah d dari 8 arah:
        JIKA tidak bisa bergerak ke d: lanjut
        nextLocation ← posisi + d
        score ← 0

        tileType ←
mapMemory[nextLocation.x][nextLocation.y]
        JIKA tileType = 0 (unknown)    : score ←
score + 100
        JIKA tileType = 1 (kosong)     : score ←
score + 50
        JIKA tileType = 3 (cat musuh)  : score ←
score + 25
        JIKA tileType = 2 (cat sekutu) : score ←
score - 75
        JIKA tileType = 4 (tembok/ruin): score ←
score - 999

        UNTUK setiap tile di visitedTiles:
            JIKA tile = nextLocation: score ← score
- 150

        score ← score + acak(0..9)
        JIKA score > maxScore:
            maxScore      ← score
            bestDirection ← d

    JIKA bestDirection ada:
        bergerak ke bestDirection
        visitedTiles[visitedIndex] ← posisi baru
        visitedIndex ← (visitedIndex + 1) mod 40
```

4.1.3.2 Struktur Data

Berikut struktur data yang digunakan pada kode program Bot Alternatif 2:

- mapMemory (array 2D integer, 61×61): Array dua dimensi yang menyimpan kondisi setiap petak peta berdasarkan koordinat (x, y). Nilai elemen merepresentasikan kondisi petak: 0 untuk petak yang belum pernah dilihat robot (unknown), 1 untuk petak kosong tanpa cat, 2 untuk petak yang sudah diwarnai tim sekutu, 3 untuk petak yang diwarnai musuh, dan 4 untuk tembok atau ruin. Struktur ini memungkinkan robot mengingat kondisi peta dari ronde-ronde sebelumnya meskipun petak tersebut sudah berada di luar jangkauan pandang.
- visitedTiles (array MapLocation, ukuran 40) + visitedIndex (integer): Circular buffer yang menyimpan 40 koordinat petak terakhir yang diinjak robot. visitedIndex berperan sebagai pointer yang menunjuk posisi mana di array yang akan ditimpa berikutnya. Ketika index mencapai 39, nilainya kembali ke 0 sehingga petak yang paling lama secara otomatis tertimpa oleh petak terbaru. Struktur ini digunakan sebagai mekanisme anti-loop pada fungsi pergerakan robot.
- alliedTowers (array MapLocation, ukuran 30) + towerCount (integer): Array yang menyimpan koordinat menara sekutu yang pernah dilihat robot. towerCount berfungsi sebagai batas atas iterasi sekaligus penunjuk slot kosong berikutnya. Digunakan oleh fungsi tryReloadPaint untuk menentukan menara terdekat sebagai tujuan pengisian ulang cat.
- knownRuins (array MapLocation, ukuran 50) + ruinCount (integer): Array yang menyimpan koordinat ruin yang pernah dilihat robot dan belum menjadi menara. Merupakan struktur data utama dari strategi Tower Rusher — soldier menggunakan daftar ini untuk secara aktif menuju ruin yang paling jauh dari menara

yang sudah ada, bukan hanya bereaksi terhadap ruin yang kebetulan terlihat.

Berikut fungsi dan prosedur yang digunakan pada kode program Bot Alternatif 2:

- run(rc): Fungsi utama yang dijalankan sepanjang hidup robot. Berisi loop tak terbatas yang setiap rondenya memanggil tiga fungsi pembaruan memori, kemudian mendelegasikan aksi ke fungsi yang sesuai berdasarkan tipe robot menggunakan struktur switch.
- updateMapMemory(rc): Memindai semua petak dalam jangkauan pandang robot dan memperbarui nilai mapMemory sesuai kondisi terkini. Dipanggil setiap ronde sebelum aksi apapun dilakukan agar robot selalu mengambil keputusan berdasarkan informasi terbaru.
- updateTowerMemory(rc): Memindai semua robot sekutu dalam jangkauan pandang dan mencatat koordinat menara yang belum tercatat ke dalam alliedTowers. Menggunakan pengecekan duplikat sebelum menambahkan entri baru.
- updateRuinMemory(rc): Memindai ruin yang terlihat dan memperbarui knownRuins. Jika ruin sudah menjadi menara (terdeteksi ada robot bertipe tower di lokasinya), ruin tersebut dihapus dari daftar menggunakan teknik swap-and-pop — elemen yang dihapus ditimpa oleh elemen terakhir array sehingga tidak perlu menggeser seluruh isi array.
- tryReloadPaint(rc) → boolean: Fungsi yang mengembalikan true jika robot sedang dalam mode pengisian ulang cat. Robot masuk mode ini ketika cat turun di bawah 30% kapasitas. Fungsi mencari menara sekutu terdekat dari alliedTowers, lalu mengarahkan robot menuju menara tersebut dan menarik cat menggunakan transferPaint dengan nilai negatif. Selama mode ini aktif, semua aksi lain dihentikan.

- runTower(rc): Mengelola logika menara dengan tiga prioritas berurutan: spawn robot, menyerang musuh, dan upgrade diri sendiri. Keputusan spawn menggunakan angka acak untuk menentukan tipe robot dengan probabilitas 40% soldier, 30% splasher, dan 30% mopper. Terdapat mekanisme fallback ke soldier apabila chips tidak mencukupi untuk tipe yang dipilih. Spawn dicoba ke semua 8 arah hingga ditemukan lokasi yang valid.
- runSoldier(rc): Mengimplementasikan logika utama strategi Tower Rusher melalui empat prioritas berurutan. Pertama, reload cat jika hampir habis. Kedua, bangun menara jika ada ruin yang langsung terlihat. Ketiga, menuju ruin terbaik di memori menggunakan heuristik greedy — dipilih ruin yang jaraknya paling jauh dari semua menara sekutu yang sudah ada, karena area tersebut paling membutuhkan spawn point baru. Keempat, eksplorasi peta jika tidak ada ruin di memori.
- buildTowerAtRuin(rc, ruinLocation): Prosedur helper yang menangani tiga tahap pembangunan menara: menandai pola 5×5 (markTowerPattern), mewarnai petak sesuai pola dengan warna primer atau sekunder yang tepat, dan menyelesaikan pembangunan (completeTowerPattern). Tahap penyelesaian dicek lebih dahulu setiap ronde agar tidak terlewat ketika pola sudah lengkap.
- paintCurrentTile(rc): Prosedur sederhana yang mewarnai petak yang sedang diinjak robot jika belum berwarna sekutu. Dipanggil oleh soldier dan splasher saat berjalan untuk memastikan setiap langkah meninggalkan jejak cat.
- runSplasher(rc): Mengelola logika splasher dengan tiga prioritas: reload cat, bergerak ke menara yang wilayah sekitarnya paling banyak belum diwarnai (menggunakan findTowerToSupport), lalu melakukan serangan AoE ke titik pusat terbaik. Titik pusat serangan dipilih secara greedy berdasarkan jumlah petak

non-sekutu yang akan terwarnai, dengan petak musuh mendapat bobot lebih tinggi (+2) dibanding petak kosong (+1).

- findTowerToSupport(rc) → MapLocation: Fungsi helper untuk splasher yang mencari menara sekutu dengan jumlah petak belum diwarnai terbanyak di sekitarnya. Hanya mengevaluasi menara yang berada dalam jangkauan sensor saat ini.
- runMopper(rc): Mengelola logika mopper dengan empat prioritas: reload darurat (hanya jika cat di bawah 10), mopper swing jika ada musuh dalam jangkauan, mengisi cat ke robot sekutu yang kritis, dan menghapus cat musuh dengan prioritas petak yang berbatasan langsung dengan wilayah sekutu.
- patrolAllyTerritory(rc): Fungsi pergerakan khusus mopper yang mengarahkannya berpatroli di sekitar wilayah sekutu. Scoring arah gerak mengutamakan petak musuh (+50), diikuti petak sekutu (+30), lalu petak kosong (+10). Dilengkapi mekanisme anti-loop dan random tiebreak.
- moveToTarget(rc, target): Fungsi navigasi untuk robot yang memiliki tujuan spesifik. Memilih arah dengan total skor terkecil dari penjumlahan jarak kuadrat ke target dan penalti anti-loop sebesar 1000 untuk petak yang baru dikunjungi. Penalti besar ini memastikan robot tidak terjebak bolak-balik namun tetap dapat menemukan jalur alternatif jika diperlukan.
- exploreMap(rc): Fungsi navigasi untuk robot yang tidak memiliki tujuan spesifik. Memilih arah dengan skor tertinggi berdasarkan kondisi petak tujuan — sangat mengutamakan petak unknown (+100) untuk memaksimalkan penemuan ruin baru, menghindari petak sekutu (-75) karena sudah tercover, dan menerapkan penalti anti-loop serta random tiebreak untuk variasi eksplorasi.

4.2 Pengujian

4.2.1 Bot Utama vs Bot Alternatif 1

4.2.1.1 Kasus 1: [Nama Map]

4.2.1.2 Kasus 2: [Nama Map]

4.2.1.3 Kasus 3: [Nama Map]

4.2.1.4 Kasus 4: [Nama Map]

4.2.1.5 Kasus 5: [Nama Map]

4.2.2 Bot Utama vs Bot Alternatif 2

4.2.2.1 Kasus 1: [Nama Map]

4.2.2.2 Kasus 2: [Nama Map]

4.2.2.3 Kasus 3: [Nama Map]

4.2.2.4 Kasus 4: [Nama Map]

4.2.2.5 Kasus 5: [Nama Map]

4.3 Analisis Hasil Pengujian

Misalnya adalah apakah strategi greedy berhasil mendapatkan nilai optimal, lalu jika tidak, dalam kondisi seperti apa strategi greedy tidak berhasil mendapatkan nilai optimal, dsb.

BAB 5
KESIMPULAN DAN SARAN

5.1 Kesimpulan

Tugas Besar 1 ini berhasil mengimplementasikan tiga bot Battlecode 2025 menggunakan pendekatan algoritma greedy dengan heuristik yang berbeda-beda. Dari hasil implementasi dan pengujian, dapat ditarik beberapa kesimpulan sebagai berikut.

1. Algoritma greedy terbukti dapat diterapkan secara efektif dalam permainan Battlecode 2025. Setiap robot mengambil keputusan terbaik berdasarkan kondisi lokal yang tersedia pada setiap ronde tanpa memerlukan perencanaan jangka panjang, yang sesuai dengan batasan bytecode yang ketat dalam permainan ini.
2. Pemilihan heuristik sangat menentukan efektivitas bot. Dari ketiga bot yang diimplementasikan, bot utama milik Alfonzo dengan heuristik berbasis pembagian kuadran, deteksi simetri peta, dan navigasi adaptif berdasarkan ukuran peta menghasilkan performa terbaik dengan coverage konsisten di atas 70%. Bot alternatif 1 yang dibuat oleh Argha dengan heuristik efisiensi cat dan fase spawn berbasis ronde menempati posisi kedua. Bot alternatif 2 milik Aurel dengan heuristik maksimasi jumlah menara menghasilkan lebih banyak menara dibandingkan bot lain, namun coverage peta secara keseluruhan lebih rendah karena soldier terlalu fokus pada pembangunan menara dan kurang mewarnai wilayah secara langsung.
3. Komposisi spawn robot berpengaruh signifikan terhadap laju pewarnaan peta. Bot yang mengadaptasi komposisi spawn berdasarkan fase permainan dan kondisi lapangan terbukti lebih efektif dibandingkan bot dengan probabilitas spawn tetap, karena kebutuhan akan tipe robot berbeda seiring berjalannya permainan.

Dengan begitu, dapat disimpulkan bahwa kelompok kami telah mengimplementasikan ketiga bot dengan heuristik yang berbeda-beda dan menghasilkan output pertandingan yang beragam pula.

5.2 Saran

Berdasarkan hasil implementasi dan pengujian, beberapa saran pengembangan yang dapat dilakukan untuk meningkatkan performa bot di masa mendatang adalah sebagai berikut.

1. Implementasi komunikasi antar-robot dapat meningkatkan koordinasi secara signifikan. Saat ini setiap robot beroperasi secara independen dengan memori lokal masing-masing. Dengan memanfaatkan sistem pesan (message) dan penanda (marker) yang tersedia dalam API Battlecode, robot dapat berbagi informasi lokasi ruin, posisi musuh, dan area yang sudah tercover sehingga seluruh pasukan dapat berkoordinasi secara lebih efisien.
2. Sistem spawn tower sebaiknya mempertimbangkan lebih banyak faktor kontekstual seperti jumlah robot aktif di lapangan, jarak rata-rata robot ke menara terdekat, dan persentase coverage peta saat ini. Pendekatan ini akan menghasilkan komposisi pasukan yang lebih adaptif dibandingkan sekadar berbasis ronde atau probabilitas tetap.
3. Soldier dapat ditingkatkan dengan menambahkan heuristik pemilihan jenis menara yang lebih cerdas. Saat ini ketiga bot selalu membangun menara cat (paint tower), padahal menara uang (money tower) dan menara pertahanan (defense tower) dapat memberikan keuntungan strategis dalam situasi tertentu, terutama di fase pertengahan dan akhir permainan.
4. Perlu dipertimbangkan strategi penanganan kondisi di mana chips menumpuk terlalu banyak tanpa digunakan, seperti yang terlihat pada bot alternatif 1 dalam beberapa pengujian. Chips yang tidak terpakai merupakan sumber daya yang terbuang dan seharusnya dapat dikonversi menjadi robot tambahan atau upgrade menara untuk mempercepat kemenangan.

Diharapkan dengan evaluasi dan saran ini, dapat meningkatkan kualitas bot dan pengalaman bermain yang lebih baik.

LAMPIRAN

Link repository Github:

https://github.com/Rizelbit/Tubes1_beeOL

Link video:

DAFTAR PUSTAKA

Laboratorium Ilmu dan Rekayasa Komputasi. (2026). Spesifikasi Tugas Besar 1 IF2211 Strategi Algoritma: Pemanfaatan Algoritma Greedy dalam Pembuatan Bot Permainan Battlecode 2025. Institut Teknologi Bandung.